SAGA LIGHT SOURCE | TECHNICAL SPECIFICATION & USER GUIDE

AILab_Note 2.0 (AIN2) 総合マニュアル & アドイン開発ガイド

シンクロトロン放射光・先端科学計測向けデジタル実験ノート & 独立アドイン拡張仕様書

Document ID	SAGA-LS_AIN2_Manual / Version 2.0.0 (Modular Edition)	Copyright	© SAGA Light Source (SAGA-LS)
Developer	SAGA Light Source (Chief Architect: Akio Yoneyama)	AI Partners	Emma, Emu & Eli

目次

- **第1章 システム概要と設計思想**
  - 1.1 AIN2の目的とコンセプト
  - 1.2 システムアーキテクチャ構成
  - 1.3 マルチアカウント完全プロファイル分離
- **第2章 インストールと環境構築**
  - 2.1 推奨動作環境
  - 2.2 詳細環境構築手順 (macOS / Windows 11)
  - 2.3 単一インスタンス排他制御と起動処理
  - 2.4 終了シーケンス・未保存検知と差分バックアップ
- **第3章 メインノート機能とUI操作**
  - 3.1 画面レイアウトと日付ナビゲーション
  - 3.2 ログの自動保存・多層保護機構 (.tmp / .bak / 差分バックアップ)
  - 3.3 放射光実験支援ツール (SR Info & Ring Current)
  - 3.4 リッチエディタ装飾とインライン・キャンバス描画
  - 3.5 ログ検索 (Find Logs)
  - 3.6 メモ帳 (Memorandum)
  - 3.7 スクラッチパッド電卓 (Calculator)
  - 3.8 Fiji / ImageJ 双方向連携 (詳細仕様)
  - 3.9 日英バイリンガルUI (i18n) & 環境設定 (Option)
- **第4章 独立アドイン製品群 (AIN2 Add-in Suite)**
  - 4.1 ブックマーク・リンク集 (Links - 01_links)
  - 4.2 定型プロトコル管理 (Protocol Manager - 02_protocol_manager)
  - 4.3 Telegram 現場メモ・写真取り込み (COP - 03_cop_telegram)
  - 4.4 AI実験パートナー「Eli」(04_eli_assistant)
- **第5章 システムアーキテクチャ & 内部仕様リファレンス**
  - 5.1 ディレクトリ構造とファイル配置
  - 5.2 バックエンド API エンドポイント一覧 (本体コア)
  - 5.3 汎用フロントエンド間通信 (BroadcastChannel)

- **第6章 アドイン開発・プラグイン仕様 (Add-in Developer Guide)**
  - 6.1 アドインの構成要件とライフサイクル
  - 6.2 manifest.json 仕様リファレンス
  - 6.3 フロントエンド連携規約 (ain_addin_bridge 実装規約)
  - 6.4 バックエンド API ルーター仕様 (router.py)
  - 6.5 最小アドイン実装チュートリアル (Tutorial)

第1章 システム概要と設計思想

1.1 AIN2の目的とコンセプト

AILab_Note 2.0 (AIN2) は、シンクロトロン放射光実験（X線CT、XAFS、微細回折・散乱など）をはじめとする過酷な先端研究現場において、刻一刻と変化する実験パラメーターや画像データを瞬時に記録・構造化するために開発されたハイブリッド型デジタル実験ノートシステムです。

- **完全ローカル完結性:** 測定ハッチ内やインターネットが遮断された高セキュリティ環境でも、全機能がミリ秒単位で高速動作します。
- **プラガブルなアドイン分離設計:** コアシステムと各種専門ツール（AI補佐、現場写真連携、プロトコル等）を完全疎結合化し、独立モジュールとして柔軟に追加・拡張可能な構成を採用しています。
- **マルチアカウント完全プロファイル分離:** 1台の測定PCを交代で使用する共同利用実験のため、最大5アカウントまでの独立プロファイル管理を標準搭載しています。

1.2 システムアーキテクチャ構成

AIN2は、Python (FastAPI) バックエンドと pywebview ネイティブシェルを統合したローカルデスクトップアプリケーションです。フロントエンドのWeb標準技術（HTML5 / CSS / Vanilla JS）とローカルHTTP/REST API（`127.0.0.1:8000`）が連携し、高速なUI応答とファイルI/O、外部解析ツール制御を両立しています。

1.3 マルチアカウント完全プロファイル分離

ビームラインや共同利用実験室では、複数の研究者や実験テーマが同一端末を共有します。ツールバー右上のアカウント選択から最大5アカウントまで即座に切り替え可能であり、すべてのログ、設定（`config.json`）、ChromaDBベクトルインデックス、Telegram写真キャッシュは `./Log/{account_name}/` 配下に完全独立して格納されます。

第2章 インストールと環境構築

2.1 推奨動作環境

項目	macOS 推奨環境	Windows 11 推奨環境
OS	macOS 13 (Ventura) 以降 (Apple Silicon / Intel)	Windows 11 64-bit (22H2 以降)
Python	Python 3.10 または 3.11 (Conda環境推奨)	Python 3.10 または 3.11
RAM / 画面	16 GB以上推奨 | 1920×1080以上	16 GB以上推奨 | 1920×1080以上
外部ツール	Fiji (ImageJ.app), Tesseract OCR (brew install)	Fiji (ImageJ-win64.exe), Tesseract OCR

2.2 詳細環境構築手順 (macOS / Windows 11)

A. macOS でのセットアップ手順

1. **Homebrew および OCRエンジンの導入:** ターミナルを開き、Homebrew と蓄積リング電流読み取り用の Tesseract OCR をインストールします。

```
# Homebrew導入 (未導入の場合)
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Tesseract OCR のインストール
brew install tesseract
```

2. **Miniconda の導入と仮想環境構築:**

```
# Python 3.11 仮想環境 'ain2' の作成とアクティベート
conda create -n ain2 python=3.11 -y
conda activate ain2
```

3. **依存ライブラリのインストール:** AIN2 ディレクトリへ移動し、各モジュールを一括導入します。

```
cd /path/to/AIN2
pip install fastapi uvicorn pydantic pywebview beautifulsoup4 requests opencv-python
numpy pytesseract xraylib chromadb
```

> **macOS pywebview に関する注記**
> macOS 環境ではシステム標準の WebKit（Safari エンジン）が直接呼び出されるため、Qtなどの重厚な追加GUIライブラリは一切不要です。

B. Windows 11 でのセットアップ手順

1. **システムツールの準備:**
   - **Tesseract OCR (Windows版):** UB-Mannheim リポジトリより `tesseract-ocr-w64-setup.exe` を入手し、`C:\Program Files\Tesseract-OCR` にインストールしてシステムの環境変数 `Path` に追加します。
   - **Microsoft C++ Build Tools:** 科学計算ライブラリ（`xraylib` 等）のビルドに必要となる場合があります。Visual Studio 公式インストーラーから導入してください。
2. **Anaconda Prompt からの環境構築:**

```
conda create -n ain2 python=3.11 -y
conda activate ain2
cd C:\path\to\AIN2
pip install fastapi uvicorn pydantic pywebview beautifulsoup4 requests opencv-python
numpy pytesseract xraylib chromadb
```

> **Windows pywebview に関する注記**
> Windows 11 では Microsoft Edge WebView2 ランタイムがOSに標準搭載されているため、追加インストールなしで高精細なモダンWebUIが動作します。

2.3 単一インスタンス排他制御と起動処理

仮想環境をアクティベートした状態で `python main.py` を実行します。二重起動防止のため、起動時にルートの `.ain_app.lock` に対してファイルロック（macOS: `fcntl.flock`、Windows: `msvcrt.locking`）を取得します。既に起動している場合は、OSのAPIを通じて既存ウィンドウを最前面に浮上させ、新規プロセスは速やかに終了します。

2.4 終了シーケンス・未保存検知と差分バックアップ

ウィンドウ終了時には `pywebview` のクローズイベントをインターセプトし、未保存変更（`isModified`）の有無を検知するモーダルが作動します。変更がある場合は「保存して終了」「破棄して終了」「キャンセル」を選択でき、設定に応じて Dropbox 等の指定フォルダへ自動差分バックアップ（`rsync --update` 方式）および一時領域（`tmp_imagej/`）のクリーンアップを実行して安全に終了します。

第3章 メインノート機能とUI操作

3.1 画面レイアウトと日付ナビゲーション

画面は左端の「ツール・アドイン起動バー」、上部の「書式・アクションヘッダー」、中央の「リッチエディタ領域（`contenteditable`）」、下部の「ステータスバー」で構成されます。ヘッダーの「◀ / ▶」で1日単位、「◀◀ / ▶▶」で実際に記録が存在するログ日への高速ジャンプが可能であり、カレンダーパネルから履歴日を視覚的に選択することもできます。

3.2 ログの自動保存・多層保護機構 (.tmp / .bak / 差分バックアップ)

編集中のテキストは、入力後デバウンス（約1.5秒）により非同期で `.tmp` ファイルへ自動退避されます。明示的な保存時および定期オートセーブ時には、既存の `YYYYMMDD.html` が `YYYYMMDD.bak` へローテーション退避された上で最新内容がアトミックに書き込まれます。さらに、設定画面で「終了時にフォルダへ差分バックアップ」を有効にしておくことで、アプリ終了時に `rsync` によりクラウドフォルダ等へ安全にミラーリングされます。

3.3 放射光実験支援ツール (SR Info & Ring Current)

- **放射光実験条件 (SR Info):** ツールバーのボタンから、施設名（SAGA-LS / SPring-8 / PF）、ビームライン名、分光結晶・フィルター、エネルギー（keV）、測定手法、参加者を設定し、整形されたメタデータテーブルをカーソル位置に挿入します。
- **蓄積リング電流値 (Ring Current):** ボタンを押すと、各放射光施設の公式サイトのモニタ画像をバックグラウンドで取得し、内蔵OCRエンジン（OpenCV + Tesseract）が電流値（mA）を自動認識してタイムスタンプ付きでノートへ記録します。

3.4 リッチエディタ装飾とインライン・キャンバス描画

1. 基本テキスト装飾ツール:

- 太字（**B**）、斜体（*I*）、下線（<u>U</u>）、取り消し線（~~S~~）、上付き文字（x^2）、下付き文字（x_2）をワンクリックでトグル。
- 文字色切替（A: デフォルト黒 / 警告・重要事項の赤色切替）、蛍光ペンマーカー（黄色ハイライト）。
- 箇条書きリスト（`<ul>`）、番号付きリスト（`<ol>`）、区切り破線、枠囲みコールアウト（Box）。

2. インライン・キャンバス描画 & 画像アノテーション:

ツールバーの描画ボタンを押すと行数指定ミニポップアップが開き、指定した高さの白紙キャンバス画像がエディタ内に即座に挿入されます。また、ノート内の任意の画像をクリックすると、青いリサイズ枠とスリム描画ツールパレットが展開します。

- **直線・矢印ツール:** 線太さスライダー（1〜10px）、線種選択（実線・点線・一点鎖線）、矢印形式（なし、右矢印→、左矢印←、両端矢印↔）、カラーピッカーによる自由な色指定。
- **図形ツール（矩形 / 円・楕円）:** 枠線太さ、線種（実線 / 破線 / 一点鎖線）、枠色、ハッチング（なし、ベタ塗り、斜線、クロス格子）、塗色指定。
- **注釈テキストツール (T):** クリック位置にプロンプト入力から文字を配置。フォントサイズ（8〜48px）、太字、斜体、下線、文字色を合成。
- **Undo / Redo & 確定保護:** アンドゥ / リドゥにより描画途中のやり直しが可能。「確定」ボタンを押すか別画像を編集しようとした際に、ベクター図形が元画像の解像度に合わせてBase64画像へ不可逆合成（焼き込み）されます。未確定のまま別操作に移る際はアラートで保護されます。

3.5 ログ検索 (Find Logs)

左サイドバーの検索ボタンから検索ダイアログを呼び出します。

- **キーワード検索:** 過去のすべての記録から本文テキストを高速全文検索。Base64画像バイナリやHTMLタグは自動的に除去され、純粋なテキストのみが対象となります。
- **日付範囲フィルタ:** 「Date range」チェックボックスを有効にし、開始日・終了日カレンダーから特定の実験ビジット期間（例: 2026-05-01〜2026-05-06）に限定した絞り込みが可能です。
- **検索結果プレビューとジャンプ:** 該当箇所が「…前文【検索語】後文…」として一覧表示され、ダブルクリックすることで該当日の実験ログがエディタへ即座に読み込まれます。

3.6 メモ帳 (Memorandum)

日付に依存しない通年の実験アイデア、薬品ストック、緊急連絡先、ビームライン光学定数などを常備するための独立サブウィンドウです。

- **独立データ構造:** `./Log/{account}/memorandum.html` に保存され、日次ログの切り替えに影響されず常に同じ内容を保持します。
- **多重バックアップ:** 起動時に既存ファイルが `memorandum.orig` に退避され、保存時には `memorandum.bak` が自動更新されるため、誤って全消去した場合でも即座に復元できます。
- **クイック装飾:** チェックボックス、箇条書き、太字、赤字切替ショートカットを搭載し、タスク管理パッドとしても機能します。

3.7 スクラッチパッド電卓 (Calculator)

左サイドバーの電卓アイコンから起動する、テキストエディタと放射光科学計算エンジンが一体化した計算パッドです。

- **計算入力規則の違い:**
  - **スクラッチパッド電卓内:** 計算専用環境のため、`?` なしで直接 `180000 div 360` や `e2ang(10, si111)` と入力して Enter を押すだけで、直下の行に計算結果が自動展開されます。
  - **メインノート内:** 日常的な文章入力との誤爆を防ぐため、先頭に `?` を付与して入力します（例: `?Fe(10)` → Enter で吸収物理量が出力）。
- **放射光光学・結晶回折計算関数:**
  - `e2w(E)`: X線エネルギー（keV）から波長（nm）を計算。
  - `w2e(lambda)`: 波長（nm）からエネルギー（keV）を計算。
  - `e2ang(E, crystal)`: 指定エネルギーと結晶面（`si111`, `si220`, `si311`, `ge111`）からブラッグ角（deg）を算出。
  - `ang2e(deg, crystal)`: ブラッグ角から回折X線エネルギーを逆算。
- **元素吸収物理量計算（xraylib 連携）:** `Fe(10)` や `Si(17.8)` と入力すると、`Fe@10keV: μ=221.73mm$^{-1}$, 1/e=0.0045mm, K-edge=7.112keV` のように線吸収係数、1/e透過厚、K吸収端エネルギーが瞬時に算出されます。
- **ヘルプチートシート（`??` 入力）:** 電卓またはメインノート上で `??` と入力して Enter を押すと、全組み込み関数（パルスモーターパルス計算 `div` / `mod`、素因数分解 `factor()`、三角関数、光学定数）のリファレンスがその場に展開されます。

3.8 Fiji / ImageJ 双方向連携 (詳細仕様)

X線CTや顕微鏡計測において、撮影画像の即時解析とノート記録をシームレスに行うための双方向IPC（プロセス間通信）ブリッジ機能です。

- **Fijiの自動起動とバックグラウンド監視:** ノート上の画像を選択して緑の顕微鏡ボタンを押すと、AIN2 バックエンドが画像を一時PNGとして書き出し、設定された Fiji を起動します。同時に Jython スクリプト（`ain_watcher.py`）がデーモンスレッドとして常駐開始し、Fiji の Recorder（操作履歴マクロ）と最前面画像ウィンドウ（`WindowManager.getCurrentImage()`）を1秒周期で自動監視します。
- **結果のワンクリック取り込み:** ユーザーが Fiji 上でコントラスト調整、スケールバー付与、粒子解析などを実行した後、AIN2 のエディタ上で右クリックして「Img from Fiji」を選択します。Fiji での最新表示画像と、実際に実行されたマクロスクリプト（例: `run("Enhance Contrast...", "saturated=0.3");`）がセットになった整形ブロックがカーソル位置へ即座に挿入されます。

3.9 日英バイリンガルUI (i18n) & 環境設定 (Option)

左下の歯車アイコンから環境設定（Option）を開けます。表示言語を「日本語」または「English (US)」に切り替えると、`TreeWalker` による動的テキスト置換と `ain_i18n_channel` 経由のブロードキャストにより、全ウィンドウのツールチップやUI表示が再起動なしでリアルタイムに切り替わります。また、ログ保存先、Fiji実行パス、自動保存間隔、終了時バックアップ先フォルダ、蓄積リング施設設定も本画面で集中管理されます。

第4章 独立アドイン製品群 (AIN2 Add-in Suite)

AIN2 では、特殊機能が `Addins/` ディレクトリ配下の独立モジュールとしてパッケージ化されています。本体側のソースコードを一切変更することなく、フォルダの追加・削除のみで機能構成を柔軟にカスタマイズできます。

4.1 ブックマーク・リンク集 (Links - 01_links)

ビームラインポータル、安全講習ページ、データ解析リポジトリ等のアクセス頻度が高いWebサイトを管理する標準アドインです。

- **機能概要:** 日常的にアクセスする実験関連リンクを登録・編集・削除できます。リンクをクリックすると OS 既定の外部ブラウザが呼び出されます。
- **完全独立設計:** 独自の設定ファイル（`links.json`）を自身のアドインフォルダ内に保持し、本体のデータベースに依存しません。
- **開発テンプレートとしての役割:** 単一の `window.html` と `manifest.json` のみで構成される最も軽量なアドインであり、サードパーティが新しい拡張を作成する際の基本スケルトンとして利用できます。

4.2 定型プロトコル管理 (Protocol Manager - 02_protocol_manager)

光学系のアライメント手順、試料チェンジャーの交換手順、キャリブレーション測定フローをカード形式で登録・編集できるアドインです。

- **手順カード編集:** プロトコル名と任意の手順ステップカード（Step 1, Step 2...）を動的に追加・並べ替え・保存できます。データはアカウント単位で独立保存されます。
- **メインノートへの一括展開:** プロトコル一覧から目的の項目をクリックすると、アドイン側でチェックボックス・箇条書き付きの整形済みHTMLカード（青色アクセントボーダー）を動的に組み立て、汎用アドインブリッジ（`ain_addin_bridge`）経由でメインエディタのカーソル位置へ瞬時に展開します。

4.3 Telegram 現場メモ・写真取り込み (COP - 03_cop_telegram)

ハッチ内やビームライン現場からスマートフォンで撮影した試料写真や速報テキストを、ケーブル接続不要で実験ノートに取り込む「COP (Common Operational Picture)」機能です。

1. **Telegram Bot の作成手順 (@BotFather):**
   - スマートフォンまたはPCで Telegram アプリを開き、`@BotFather` を検索してチャットを開始（`/start`）します。
   - `/newbot` を送信し、画面の指示に従って Bot の表示名とユーザー名（末尾が `bot` で終わる英数字）を決定します。
   - 発行された HTTP API Token（例: `7123456789:AAFnX...`）をコピーします。
2. **許可ユーザーIDの確認 (@userinfobot):** 第三者による不正投稿を遮断するため、`@userinfobot` にメッセージを送り、自身の数値ID（例: `123456789`）を取得します。
3. **アドイン内設定パネルでの登録:** アドイン画面右上の歯車アイコンを開き、取得した `Bot Token` と `Allowed User IDs`（カンマ区切りで複数指定可）を入力して保存します。
4. **現場からの取り込み運用:** 現場から Bot 宛てに写真や気付きメモを送信します。AIN2 側で本アドインを開くと過去7日間のメッセージがタイムライン順（新しい順）にカード表示され、未取込のカードをクリックするだけで写真（幅380pxリサイズ・キャプション付き）またはメモテキストがメインノートへ直接挿入されます。取り込まれたカードは「挿入済」として視覚的にトーンダウンします。

4.4 AI実験パートナー「Eli」 (04_eli_assistant)

Google Gemini 3.8 Flash やローカル vLLM サーバー（Mistral 等）と連携する専属AIアシスタントです。「チャット」「Log整形」「RAG生成」の3大タブを備えています。

- **接続設定とマルチモデル対応:** OpenAI 互換エンドポイント規格に準拠しており、クラウド（Google AI Studio Gemini 3.8 Flash）およびローカル GPU サーバー（vLLM / Ollama）を自由に切り替え可能です。設定パネルの「取得」ボタンから接続先で稼働中のモデル一覧をワンクリックで読み込めます。
- **チャットモード（連続対話・単発・ノート引用ディスカッション）:**
  - **連続会話:** デフォルトで有効。直前6往復分の会話履歴をバックエンドへ引き渡し、文脈を考慮した深い考察を行います。
  - **ノート引用質問:** メインノート上でテキストを範囲選択した状態で Eli を起動すると、「引用中: …」として自動的にプロンプト枠へ格納され、「この測定データの異常値の原因は？」といったピンポイントな質問が即座に行えます。
  - **回答のノート反映:** 回答バブル下部の「AINノートに挿入」を押すと、整形された Eli の考察カードがメインノートへ直接書き込まれます。
- **生ログの構造化（eLog整形ワークフロー）:** 日々の生ログ（HTML）からノイズ（Base64画像バイナリや無駄なタグ）を除去し、Frontmatter（日付、施設、BL、Energy、手法）と【実験条件】【測定手順・操作ログ】【考察・ToDo】に整理された Markdown（eLog）を生成します。

- ◦ **表示モード切替:** 「ログ: ソース」で原文を確認し、「ログ: 画像除去」に切り替えると Base64 画像が `[画像]` に置換されたクリーンなテキストを抽出できます。
  - ◦ **柔軟な2系統の整形ルート:** 「選択ログを整形」ボタンを押して内蔵 Eli に一発要約させるルートに加え、画像除去テキストをコピーしてブラウザの外部AI（Web版 Gemini やエマ）に渡し、出力された Markdown をエディタ枠に貼り付けて「./eLog へ保存」するハイブリッド運用にも完全対応しています。
- **ローカル ChromaDB による RAG 知識ベース:** 保存された `eLog/{account}/*_elog.md` を知識ベースとして、ローカルの ChromaDB にベクトル埋め込み（Embedding）を行います。
  - ◦ **差分同期:** 日常の実験終了後、「新規elogを差分追加」を押すことで、未登録のログ日付だけを数秒で高速追加します。
  - ◦ **全再構築（クリーン）:** プロンプトやチャンク分割ルールを刷新した際、既存DBを全初期化して全期間のログからインデックスを再作成します。
  - ◦ **自動検索連動:** チャット質問時に、クエリに関連する過去の実験チャンク（測定条件やトラブルシューティング記録）が自動検索され、プロンプトへ参考情報として動的注入されます。

第5章 システムアーキテクチャ & 内部仕様リファレンス

5.1 ディレクトリ構造とファイル配置

```
AIN2/
├── main.py                      # アプリ起動・pywebviewウィンドウ管理・排他制御・バックアップ
├── core/                        # コア基盤モジュール
│   ├── __init__.py
│   ├── addin_loader.py          # Addinsフォルダ動的スキャン・ルーター登録
│   └── config.py                # アカウント別設定CRUD・ウィンドウ座標永続化
├── routers/                     # 本体コアAPI
│   ├── logs.py                  # 日次ログ (.html/.tmp/.bak)、検索、共通メモAPI
│   ├── imagej.py                # Fiji連携・一時同期画像処理API
│   └── tools.py                 # リング電流OCR、ファイル/フォルダ選択ダイアログ
├── templates/                   # 本体画面HTML (index.html, memo.html, calc.html)
├── static/                      # 静的アセット (css/, icon/, img/, js/)
│   └── js/                      # ain_main.js, ain_i18n.js, ain_calc_engine.js,
ain_canvas.js
├── Addins/                      # 独立アドインモジュール群
│   ├── 01_links/                # ブックマーク・リンク集
│   ├── 02_protocol_manager/     # 定型プロトコル管理
│   ├── 03_cop_telegram/         # Telegram現場写真連携
│   └── 04_eli_assistant/        # AI実験パートナー (Eli)
└── Log/{account}/               # 実験データ格納ルート (アカウント別完全分離)
```

5.2 バックエンド API エンドポイント一覧 (本体コア)

エンドポイント	メソッド	機能・役割
`/api/logs/{date_str}`	GET / POST	指定日の生HTMLログ取得およびアトミック保存（`.bak` 退避付き）
`/api/logs_dates`	GET	ログが存在する日付（YYYYMMDD）の一覧配列を取得
`/api/search`	POST	本文テキストの全文キーワード検索および日付範囲フィルタリング

エンドポイント	メソッド	機能・役割
`/api/memo`	GET / POST	共通メモ帳（`memorandum.html`）の取得および保存
`/api/config`	GET / POST	現在アクティブなアカウントの設定ファイル（`config.json`）の読み書き
`/api/accounts`	GET / POST	登録アカウント一覧取得、アカウント切り替え、新規作成
`/api/set_window_title`	POST	pywebview 独立サブウィンドウのOSネイティブタイトルバー更新

5.3 汎用フロントエンド間通信 (BroadcastChannel)

本体エディタとアドイン画面は、HTML5 標準の BroadcastChannel を介して安全に疎結合通信を行います。本体側には特定のアドイン専用ロジックを持たせず、汎用ブリッジを通じてテキストやHTMLをやり取りします。

チャンネル名	主なアクション	ペイロードと動作

`ain_addin_bridge`	`insert_html`
	`insert_text`
	`request_selection`
		アドイン側から整形済みHTMLまたはプレーンテキストをカーソル位置へ挿入。または本体エディタの選択範囲テキストの吸い出し。

`ain_i18n_channel`	`change_lang`	`{ action: 'change_lang', lang: 'ja' | 'en' }` を送受信し、全ウィンドウの言語表示をリアルタイム同期。

第6章 アドイン開発・プラグイン仕様 (Add-in Developer Guide)

開発者は、独自のPython解析スクリプトやWebインターフェースを極めてシンプルな手順で AIN2 の正規アドインとしてパッケージ化し、導入することができます。

6.1 アドインの構成要件とライフサイクル

`Addins/` ディレクトリ配下に、ソート順プレフィックス付きのフォルダ（例: `05_my_sensor/`）を作成します。起動時に本体の `core.addin_loader` がフォルダを自動スキャンし、以下の3ファイルを認識してロードします。

- `manifest.json`：アドインの識別名、UIタイトル、アイコン、ウィンドウサイズ等を定義する設定書。
- `router.py`：バックエンドで実行される FastAPI の `APIRouter` 定義（省略可能）。
- `window.html`：独立サブウィンドウとして開かれるフロントエンド画面。

6.2 manifest.json 仕様リファレンス

```
{
  "id": "my_sample_tool",
  "name": "サンプル計測ツール",
  "name_en": "Sample Measurement Tool",
  "description": "外部センサーから値を取得してノートに記録します",
  "version": "1.0.0",
  "order": 5,
  "icon": "<svg viewBox=\"0 0 24 24\" fill=\"none\"><circle cx=\"12\" cy=\"12\" r=\"9\"
stroke=\"currentColor\" stroke-width=\"2\"/></svg>",
  "window": {
    "title": "AIN Tool - Sample",
    "width": 600,
    "height": 700,
    "resizable": true
  }
}
```

※ `icon` キーには SVG 文字列を直接記述するか、同フォルダ内の画像ファイル名（例: `"icon": "sensor.png"`）を指定できます。

6.3 フロントエンド連携規約 (ain_addin_bridge 実装規約)

アドイン側の JavaScript からメインノートへデータを差し込む、または選択テキストを取得するには、以下の定型コードを記述します。

```
// 1. メインノートのカーソル位置へ整形済みHTMLを挿入する
const bridge = new BroadcastChannel('ain_addin_bridge');

function sendResultToNotebook(title, value) {
    const html = '<div class="box-wrapper" style="border-left: 4px solid #0d9488;
background-color: #f0fdfa; padding: 8px 12px; margin: 6px 0; font-size: 13px;">'
               + '<b>' + title + ':</b> <span style="font-family: monospace;">' + value +
'</span>'
               + '</div><br>';
    bridge.postMessage({ action: 'insert_html', html: html });
}

// 2. 現在ノート上で選択中のテキストを要求・取得する
bridge.postMessage({ action: 'request_selection' });
bridge.onmessage = (event) => {
    if (event.data.action === 'response_selection') {
        console.log("ノートから取得した選択文字列:", event.data.text);
    }
};
```

6.4 バックエンド API ルーター仕様 (router.py)

アドイン独自のデータ保存やハードウェア通信を実装する場合、`router.py` に `router = APIRouter()` を定義します。本体側へは自動的に `/api/addin/{フォルダ名}/...` というプレフィックスでマウントされます。

```
from fastapi import APIRouter
from pydantic import BaseModel

router = APIRouter()

class SensorPayload(BaseModel):
    channel: int

@router.post("/read_value")
def read_sensor_data(payload: SensorPayload):
    # シリアル通信や計算ライブラリの呼び出し
    measured_val = 123.456
    return {"status": "ok", "channel": payload.channel, "value": measured_val}
```

6.5 最小アドイン実装チュートリアル (Tutorial)

もっともシンプルなアドインは、`manifest.json` と `window.html` の2ファイルだけで構成できます。`01_links`（ブックマーク集）を複製し、`manifest.json` の `id` と `name` を書き換えるだけで、即座にサイドバーへ新しいツールアイコンが追加されます。本体のPythonコードを1行も触ることなく、完全に独立した配布用プラグインを作成することが可能です。

> **アドインのパッケージ配布に関する備考**
> 作成したアドインフォルダは、そのままZIPアーカイブとして配布できます。ユーザーは AIN2 の `Addins/` フォルダに展開してアプリを再起動するだけで、即座に新しい機能を利用開始できます。